Marine Geodesy, 37:267–281, 2014
Copyright © Taylor & Francis Group, LLC
ISSN: 0149-0419 print / 1521-060X online
DOI: 10.1080/01490419.2014.902884

Delineating Sea Level Rise Inundation Using a Graph Traversal Algorithm

XINGONG LI,[1] C. J. GRADY,[1,2]
AND A. TOWNSEND PETERSON[2]

[1]Department of Geography, University of Kansas, Lawrence, Kansas, USA
[2]Biodiversity Institute, University of Kansas, Lawrence, Kansas, USA

The rate of sea level rise will roughly double over the next century with a conservative projection of 0.18–0.59 m by 2100 (IPCC 2007). Inundation caused by sea level rise will likely disrupt the physical, economic, and social systems in coastal regions worldwide. All existing methods to assess the potential impacts of sea level rise could only delineate the inundation by a specific sea level rise. In addition to providing another inundation method based on the cost distance GIS function, this research developed an innovative method that calculated the minimum sea level rise needed to inundate each cell in a Digital Elevation Model (DEM). Both methods considered water connectivity and performed better than the simple "bathtub" approach, especially with sea level rises below 1 m. Implementation data structures significantly affected the efficiency of the new method. Several data structures were proposed and compared. Our results indicated the combination of a binary heap and hash table data structure was the most efficient implementation.

Keywords Dijkstra, binary heap, cost distance, GIS

Introduction

During the 20th century, world sea levels rose by 0.17 ± 0.05 m (IPCC 2007). The Intergovernmental Panel on Climate Change (IPCC) estimates that the rate of sea level rise will roughly double over the next century due to increasing global temperatures, with a conservative projection of global sea level rise of 0.18–0.59 m by 2100 (IPCC 2007). The IPCC report, however, acknowledges that its sea level rise appraisal does not take into account the recent rapid changes to the ice sheets that have been observed since 2003. The Greenland Ice Sheet contains a volume of water equivalent to 6 m of sea level rise, and the West Antarctic Ice Sheet, an unstable ice mass grounded well below sea level, contains a volume of water equivalent to 5 m of sea level rise (Bindschadler 1998). Both the Greenland Ice Sheet and the West Antarctic Ice Sheet are currently showing rapid increases in mass loss that may significantly increase sea level if such mass loss continues (Thomas et al. 2004; Rignot and Kanagaratam 2006). Overpeck et al. (2006) indicated that warming polar

Received 20 September 2013; accepted 6 March 2014.
Address correspondence to Xingong Li, 1475 Jayhawk Blvd., Department of Geography, University of Kansas, Lawrence, KS 66045. E-mail: lixi@ku.edu
Color versions of one or more of the figures in the article can be found online at www.tandfonline.com/umgd.

temperature may reach a level this century similar to that of the Last Interglaciation (LIG), which led to sea levels about 4 to 6 m higher than present.

Nearly a quarter of the world's population lives at elevations below 100 m and within 100 km from a coast (Nicholls et al. 2011). Coastal regions also have the greatest concentration of economic activities. Inundation caused by sea level rise will likely disrupt the physical processes, economic activities, and social systems in coastal regions around the world (Nicholls and Tol 2006). Numerous assessments of present and future coastal impacts of sea level rise have been conducted at global, regional, and local scales with different focuses on population, land use/cover, flora, fauna, and biodiversity (Nicholls and Tol 2006; Hopkinson et al. 2008; Legra et al. 2008; Li et al. 2009; Craft et al. 2009; Virah-Sawmy et al. 2009; Menon et al. 2010; Peterson et al. 2010). One of the key requirements in those analyses is the delineation of potentially inundated areas by different sea level rises (SLRs). Delineation of inundation from SLR differs from existing event flood models that describe short-term, pulsed-flood events (Poulter and Halpin 2008). These flood models depend on parameters that include surface roughness coefficients and the magnitude and duration of the flooding (Marks and Bates 2000). The long-term and near steady-state inundation resulting from SLR is less dependent on surface roughness features and individual storm characteristics, and research is needed to improve our understanding on hydraulic characteristics of coastal flooding from SLR (Poulter and Halpin 2008).

Inundation by a specific SLR could be simply identified as the areas whose elevation is below or equal to the SLR. Although this "bathtub" approach is simple, it has one important shortcoming; that is, water connectivity is not considered when delineating the inundation. It is possible that there are areas that have an elevation below the SLR, but they will not be inundated because terrain barriers exist between them and ocean water. Several inundation delineation methods that do consider water connectivity have been proposed (Poulter and Halpin 2008; Li et al. 2009). Li et al. (2009) discussed two methods of delineating inundation by a specific SLR. One of the methods uses the region group GIS function (also called component labeling in image processing) to delineate spatially contiguous inundation. The other method is similar to the method used by Poulter and Halpin (2008), where water propagates from current oceans and inundates the cells whose elevation is below a specific SLR. While the efficiency of their method was not documented in Poulter and Halpin (2008), Li et al. (2009) found the method significantly slower than the method based on the region group GIS function. One common limitation among those existing methods is that they can only delineate the inundation by a specific SLR. Although those methods could be called multiple times to compile a map of inundation by several SLRs, this approach is not efficient, and, more importantly, does not take the full advantage of the vertical resolution, which continues to improve, of the DEM.

The research reported here first proposed yet another method of delineating inundation by a specific SLR. The method used the cost distance GIS function and could be easily implemented in any raster GIS having the function available. Inspired by the cost-distance based method, this research also developed an innovative method that calculated the minimum SLR needed to inundate each cell in a DEM. Once calculated, this inundation height raster layer can be used to query the inundation by any SLRs. Several data structures were proposed and compared for efficient implementation of the method.

Methods

While the "bathtub" method simply selects the cells with elevations less than or equal to a specific SLR as the inundation, methods that consider water connectivity only include

those cells in inundation if their elevation is less than or equal to the SLR and can be reached by ocean water through spatial propagation. Two methods are proposed in this research, both considering water connectivity through either 4- or 8-neighbors. The first method delineates inundation by a specific SLR while the second method calculates the minimum SLR to inundate each cell in a DEM.

Inundation by a Specific Sea Level Rise

Li et al. (2008) used the region group GIS function to test whether the cells under a specific SLR are spatially connected to oceans. As illustrated in Figure 1, our first method here

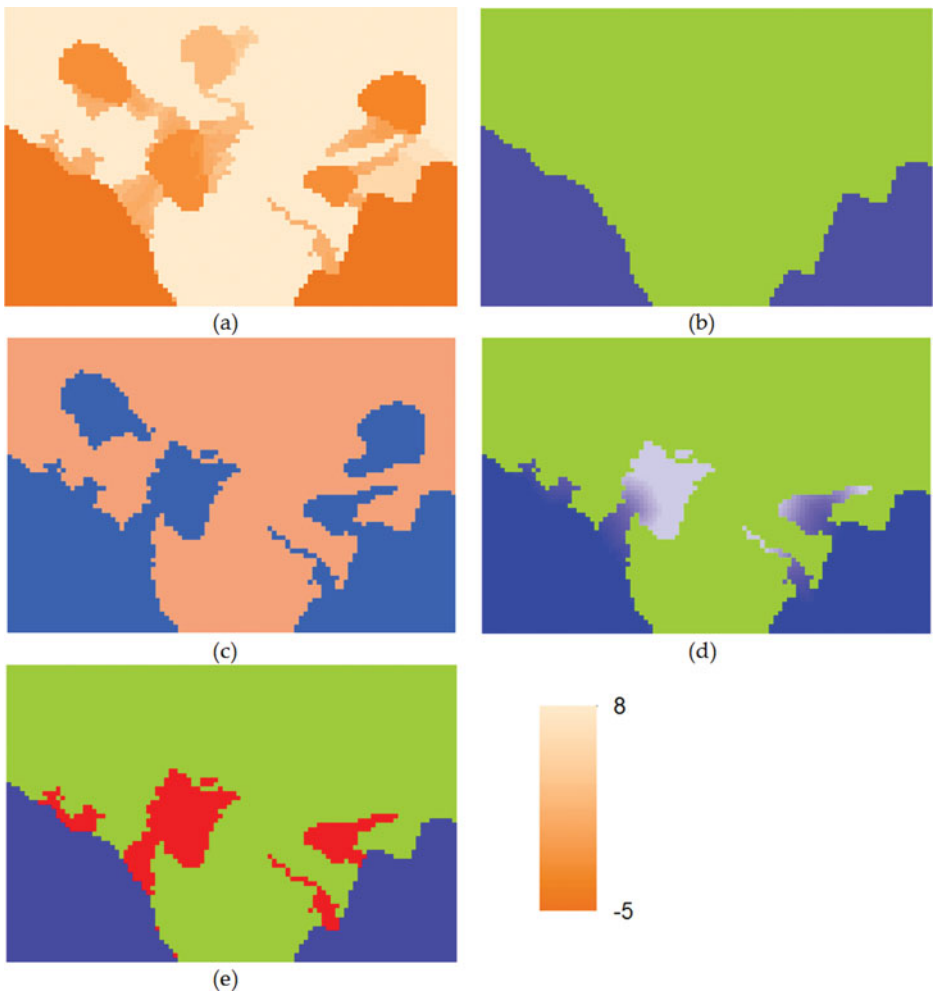

Figure 1. Delineating inundation by a specific sea level rise using the cost distance GIS function. (a) An example DEM with elevation ranging from −5 to 8 m; (b) current oceans (blue) and land (green); (c) the friction surface where blue cells are less than or equal to 2 m and have a friction of 1 and brown cells are physical barriers; (d) shortest distance calculated by the cost distance function where light blue indicates farther away from current oceans; (e) inundation by 2 m SLR (red).

used the cost distance GIS function to delineate spatially connected cells under a specific SLR. The cost distance GIS function calculates the least cost from each cell to some source cells (i.e., cells to which least cost is calculated) based on a friction surface. The function requires two input raster layers, one for the friction surface and the other one for the source cells. For the purpose of delineating the inundation by a specific SLR, current ocean cells (Figure 1b) are treated as the source cells from which ocean water propagates. The friction surface (Figure 1c) is created by setting a friction of 1 to the cells whose elevation is below and equal to the SLR and physical barriers to the cells whose elevation is above the SLR. With the source and friction raster layers thus created, the cost distance GIS function can then be used to calculate the least cost distance (Figure 1d) from current oceans to the cells to which ocean water can "travel", that is, the cells that are not blocked by any physical barriers (i.e., terrain that is above the SLR). The inundation cells (Figure 1e), therefore, are the cells whose distance to the current oceans is greater than zero (current ocean cells have the distance of zero because they are the source cells). Figure 2 illustrates how this method can be implemented as a ModelBuilder© model with the Esri ArcGIS© software.

Inundation Height

Our second method was inspired by the cost distance function in the first method. Instead of just taking advantage of the implicit spatial propagation in the cost distance GIS function, we developed a method that calculates the minimum SLR needed to inundate each cell in a DEM. We call the minimum SLR inundation height.

Our method is based on the Dijkstra algorithm that is a graph traversal algorithm that finds the minimum cost and path on a graph with nonnegative costs (Dijkstra 1959). It begins at a start node and adds all nodes directly connected to it to a set of possible nodes (front nodes hereafter). Then, the node that has the least cost in the front nodes is extracted and added to the set of visited nodes. All the nodes connected to the extracted node are added to the front nodes, and any nodes in the front that can be reached (through the extracted node) with less cost than before have their costs reduced. The process repeats until all nodes have been visited.

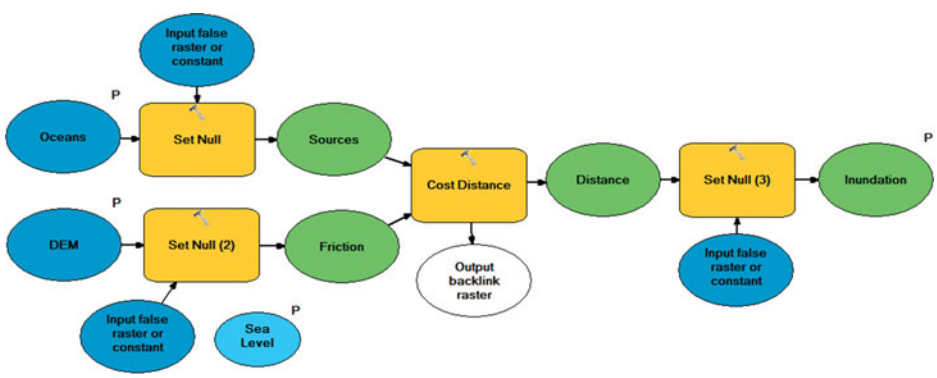

Figure 2. Cost distance based inundation model built using ModelBuilder of the ArcGIS software. Rectangles are GIS operations. The blue and green ellipses connected to the rectangles are, respectively, the inputs and outputs of the GIS operations. Ellipses with a letter "P" indicate the parameters of the model.

Dijkstra's algorithm can be adapted for computing inundation height for a DEM raster layer by treating each cell center as a node and creating edges between each cell and its neighbors. Our method supports both 4- and 8-neighbor connectivity, although all the analyses were based on 8-neighbor connectivity. With this graph view on a raster layer, current ocean cells are the start nodes and the algorithm iteratively spreads from the ocean cells by maintaining a set of land cells that are the immediate neighbors of ocean cells. We call this set of land cells the *inundation front* hereafter. Initially, the inundation front contains all the land cells next to current ocean cells with their elevation as their inundation height. In each iteration, the algorithm extracts from the inundation front the cell with the lowest inundation height (low-cell hereafter), writes the inundation height on the output raster layer, and updates the inundation front by finding the low-cell's neighbor cells, calculating their inundation heights, and inserting those cells into the inundation front. When there are several cells having the same lowest inundation height in the inundation front, the low-cell is the cell that enters the inundation front the first. The inundation height for a cell is calculated as:

$$\text{Inundation height} = max\ (\text{low} - \text{cell}'\text{s inundation height, elevation of the cell}) \quad (1)$$

The above process repeats until all the cells have their inundation height calculated. Figure 3 gives an example DEM (Figure 3a) and a current ocean (Figure 3b) raster layer from which inundation height is calculated. The example DEM has six rows and six columns and the ocean layer has three current ocean cells. Figures 3c shows the initial cells in the inundation front and their inundation heights. Figures 3d–j illustrate the snapshots of running the algorithm a few more iterations. The primary output of the algorithm is a raster layer where the value at a cell represents the minimum SLR required to inundate the cell. A possible secondary product is a raster layer that stores the shortest path by which a cell is inundated by ocean water.

Data Structures

The key data structure in the algorithm is to represent and manage the inundation front from which ocean water propagates. Any data structure used to manage the inundation front must provide two operations: the *extraction operation*, which finds the cell with the lowest inundation height in the front and removes it from the front; and the *insertion operation*, which either adds a new cell into the front if the cell does not exist in the front or updates the cell's inundation height if necessary. The front expands and shrinks during its propagation. As the size of the raster layer increases, so does the number of cells in the front, and the number of iterations will increase quadratically. This means the extraction and insertion operations will be called more times with more cells in the front. Designing efficient data structures for the operations on the inundation front becomes the most crucial component of implementing the algorithm.

The front behaves like a minimum priority queue (Cormen et al. 2001), where each item in the queue has an associated priority, which in our case is inundation height. One way to build a minimum priority queue is to use a binary heap. A binary heap is a *complete binary tree* that has as many nodes as it can hold at all levels except for the leaves. Any nodes on the leaves are pushed to the left. In a minimum binary heap (min-heap hereafter), each tree node is, at most, as large as its two children. While the two children must be at least as large as their parent, either may be larger than the other. Because each node is at most as large as the two nodes below, the root node is always the smallest node in

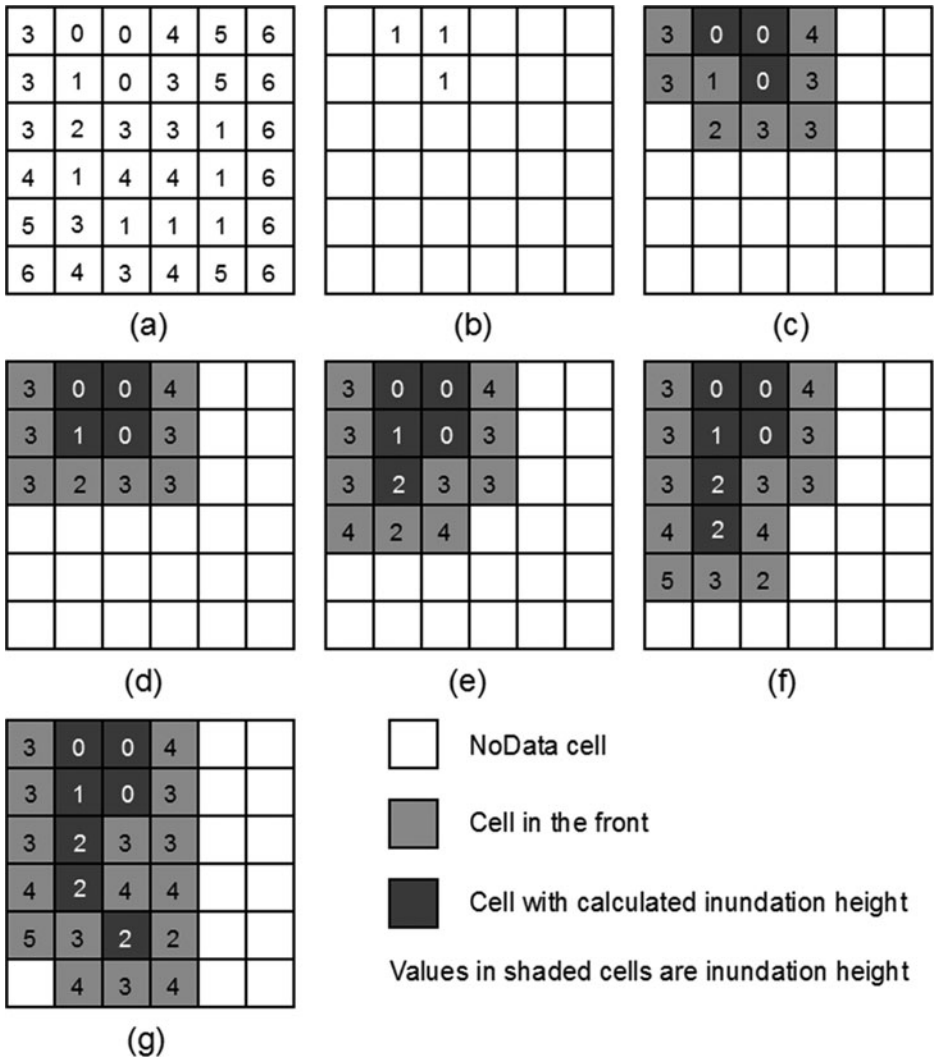

Figure 3. Calculating inundation height on an example DEM. (a) The DEM; (b) current ocean raster layer with 3 ocean cells; (c) initial inundation front and their inundation height; (d)–(g) inundation height calculation for four cells and the evolution of the inundation front.

a min-heap. This makes min-heaps a good data structure for implementing a minimum priority queue. A binary heap also has several important properties. First, it is the shortest tree that can hold a given number of nodes. Second, if a binary heap contains N nodes, its height is log(N). Last, a binary heap can be stored very compactly in an array.

To remove the smallest items from a min-heap, the last item is moved to the top of the tree. The item is then pushed down until it reaches its final position and the tree is again a valid min-heap. Because the tree has a height of log(N), this process can take, at most, log(N) steps. This means a min-heap based priority queue takes $\Theta(\log N)$ run time to extract its smallest item. To add a new item to a min-heap, the item is placed at the bottom of the tree and then pushed upward until the tree is again a valid min-heap. Because the tree has

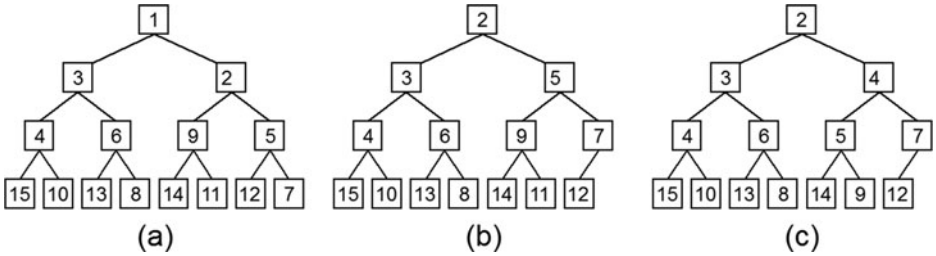

Figure 4. An example min-heap and some operations on the heap. (a) The min-heap after items 15, 14, 13, 9, 10, 12, 4, 3, 1, 8, 6, 7, 11, 2, and 5 are inserted into the heap; (b) the min-heap after item 1 is extracted; (c) the min-heap after item 11's priority in (b) is decreased to 4.

a height of log(N), this process can take, at most, Θ(log N) run time. Decreasing an item's priority is similar to adding a new item. After the priority of the item is reduced, the item is pushed upward until the tree is again a valid min-heap. This will take, at most, Θ(log N) run time too. Figures 4b and 4c give the examples of extracting a node and decreasing the priority of a node in a min-heap.

Another way to build a minimum priority queue is to use a Fibonacci heap (Fredman and Tarjan 1987). A Fibonacci heap is a more complicated data structure than a binary heap but may provide potential performance gains. Figure 5a shows an example Fibonacci heap. In a Fibonacci heap, every node is a sub-heap and each node has four pointers: one to its parent (null if it is a root node), one to its left sibling, one to its right sibling, and one to its child with the smallest key. Each node has a key, which is the value used to determine its position in the heap, which, in our case, is inundation height. Additionally, each node has a rank that indicates how tall the sub-heap is at that location and a bit indicating if the node has been marked, meaning it has lost a child.

The insert operation for a Fibonacci heap creates a new sub-heap and inserts it into the heap's ring of root sub-heaps and performs a merge operation. If the newly inserted item has the lowest key for the entire heap, the minimum pointer changes to point at it and the operation is complete. The efficiency of this operation is Θ(1) and runs in constant time. Figure 5b illustrates the heap after a new item (7) is inserted into the heap shown in Figure 5a.

To remove the smallest item from a Fibonacci heap, the children of the item pointed at by the minimum pointer are added to the root ring of sub-heaps. In the worst case scenario, this can result in all items being in the root ring of sub-heaps and becoming very inefficient. To combat this, when the extract operation takes place, sub-heaps with the same rank are linked together by making one a child of the other so that the heap property is maintained. If a node is marked, it is moved to the root level. Once all of the sub-heaps are linked, the minimum pointer is set to the sub-heap with the item with the smallest key. The amortized cost of this operation is O(log N). The operation is demonstrated in Figures 5c–f, where the minimum item (2) is deleted by first moving the root minimum pointer to the sibling on the right (10). Then, the children of node (2) are promoted to the root level of the heap. From there, link operations are performed on sub-heaps with the same rank until no two sub-heaps at the root level have the same rank. At that point, the root minimum pointer is reset to the smallest root item (3).

To reduce an item's key in a Fibonacci heap is similar to insert an item. First, the key of the item in question is reduced. If the heap property is no longer satisfied, the parent

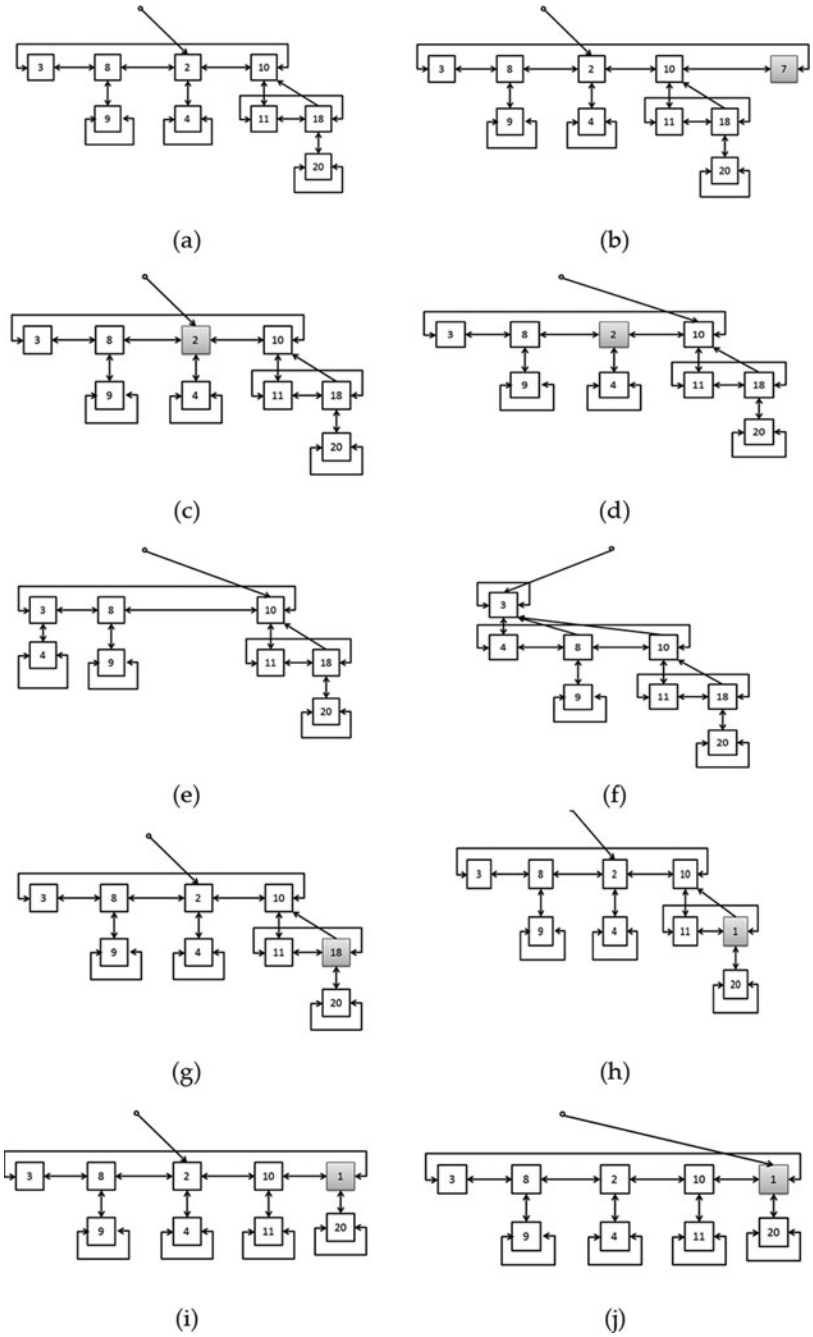

Figure 5. An example Fibonacci heap and operations on the heap. (a) The original Fibonacci heap; (b) the heap after item 7 is inserted; (c)–(f) the extract minimum operation. The minimum pointer is moved to a sibling and the minimum node's children are promoted to the root level. Sub-heaps with the same degree are merged until there root level no longer contains any two sub-heaps with the same degree (height); (g)–(j) the Reduce Key operation. Item (18) has its key reduced and is cut from its parent and moved to the root level. Then the minimum pointer is reset to the new heap minimum.

Table 1
Theoretical computational complexity of the three main operations in the heap implementation of a minimum priority queue

Operations	Binary Heap	Fibonacci Heap
Insert an Item	$\Theta(\log N)$	$\Theta(1)$
Remove the Minimum Item	$\Theta(\log N)$	$O(\log N)$
Reduce Key	$\Theta(\log N)$	$\Theta(1)$

of the item is marked and the entire sub-heap is cut from its parent and inserted at the top level of the heap. From there, the same procedure used for insert an item is performed. The amortized efficiency of this operation is $\theta(1)$. Figures 5g–j demonstrate the process, where item 18 has its key reduced to 1. The parent of the item 18 is marked and the entire sub-heap is cut and moved to the root level. Finally, the root minimum pointer is reset to item 1.

The data structures used to implement the minimum priority queue can greatly influence the performance of the method. Both binary heap and Fibonacci heap have advantages and drawbacks. While binary heaps are simpler to implement, Fibonacci heaps provide better theoretical computational complexity as shown in Table 1, where N is the number of items in a heap.

Whether a min-heap or a Fibonacci heap is used to implement the minimum priority queue, a crucial step in the insertion operation is to determine whether a low-cell's neighbor cells are already in the inundation front before adding them to the front or updating their inundation height. The operation has to search through the front based on the location of a cell. Depending on how the spatial connectivity is defined, this operation is called four or eight times in each iteration. As the number of cells in the front increases, this operation becomes a bottleneck if a sequential search is performed. Therefore, we built a location index for front cells using a hash table. With the hash table index, the average time to access a cell in the front is $\Theta(1)$ and the worst-case time is $\Theta(N)$ where N is the number of cells in the front.

Our min-heap implementation used a dynamic array which expands and shrinks as cells are inserted and extracted from it (Figure 6). An item in the array is a pointer to a front cell object. Each front cell object stores inundation height, a linear index, and its position index in the min-heap array. Inundation height is the key in the min-heap. A hash table is used to create a location index to front cell objects. Each hash table node stores a location key, which is the linear index of a cell on a DEM raster layer, and a pointer to a front cell object. Maintaining a position index in a front cell object adds extra complexity to the extraction and insertion operations which move front cells in the min-heap array. However, this effort is rewarded when the insertion operation has to quickly find a cell in the min-heap array by its location index, and when the inundation height of a cell needs to be decreased.

The two heaps and the hash table data structures were implemented in C^{++} and wrapped to work with MATLAB. All the calculations were performed using a Lenovo Thinkpad x220t machine with a 2.70 Intel Core i7-2620M processor with 8 GB RAM running MATLAB R2007b.

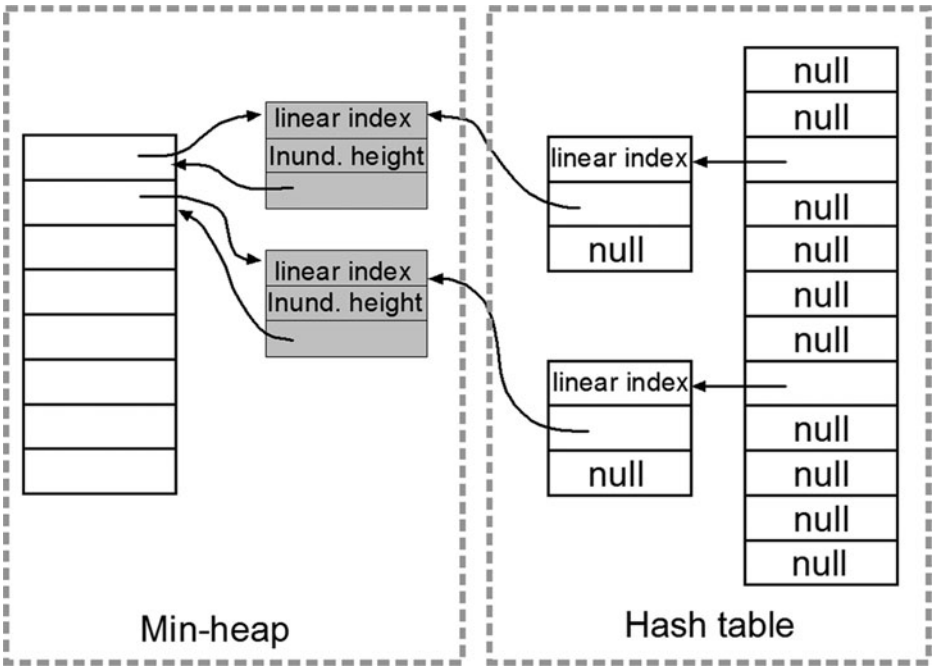

Figure 6. Data structures used in the min-heap implementation of the inundation height method.

Data

To test our methods and to evaluate the efficiency of different implementation data structures, we used a DEM covering the east coast zone of North Carolina (Figure 7). The DEM was downloaded from the USGS National Elevation Dataset (NED) website. Several individual tiles of the DEMs were downloaded then pieced together to cover the entire region at a spatial resolution of 30 m. Since LiDAR data were used to generate the DEM, it has high vertical accuracy and resolution. The DEM grid has a dimension of 9188 $\times$ 9360 (about 86 million) cells. Among them, 54% are land cells, which have a mean elevation of 14.83 m and range between -0.12 m and 121.36 m.

Results and Discussions

To compare the simple "bathtub" approach and our methods, we used Eq. (2) to calculate an error percentage. We used the equation and the term "error" here because the "bathtub" approach overestimates inundation and our methods are theoretically correct, and the true inundation is not available.

$$error\,\% = \frac{Bathtub\,Inundation - Our\,Inundation}{Our\,Inundation} \tag{2}$$

Figure 8 shows the error percentages calculated using Eq. (2) with 28 SLRs. The error percentage decreases from 48% to 0.3% as SLR increases from 0.1 to 10 m. The average error percentage for the SLRs of less than 1 m is 17%. However, the error percentage drops drastically to 1.5% when SLR reaches 1 m and reduces to an average of 0.6% for

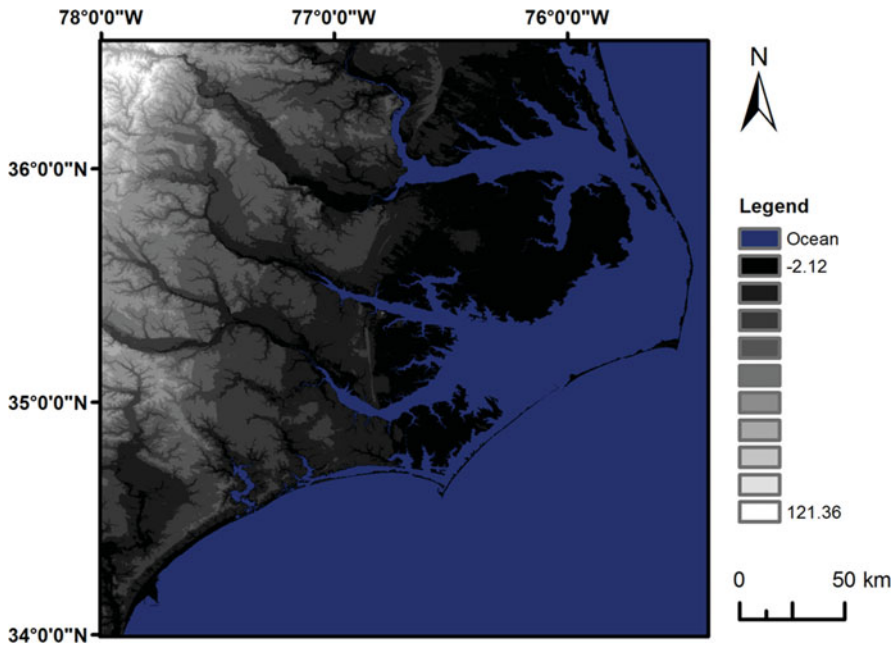

Figure 7. A DEM covering the east coast of the state of North Carolina, USA. Ocean is in blue and elevation is in shades of gray, the darker the lower.

SLRs above 1 m. With IPCC's projection of global SLR of 0.18−0.59 m by 2100, the error percentage ranges between 40% and 6% on the North Carolina DEM. Those results indicate that it is especially important to consider water connectivity in delineating inundation with small and conservative SLR scenarios.

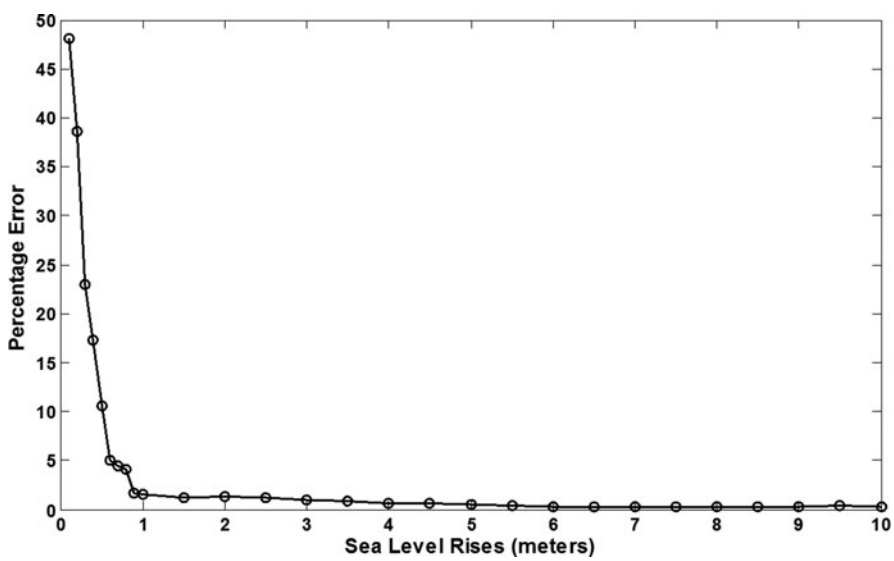

Figure 8. Percentage error of the "bathtub" method with different sea level rises.

To see how the choice of data structures may affect the efficiency of calculating inundation height, we also implemented the method using only matrix functions (i.e., no heap and hash table data structures) in MATLAB to perform the operations (pure-MATLAB implementation hereafter). Figure 9 shows the running time for an implementation using a binary heap and hash table and the pure-MATLAB implementation with eight different sizes of sub-DEMs from the North Carolina DEM. The sub-DEMs have the land cells range from 197,000 to 8 million. With the increase of land cells on those sub-DEMs, running time also increased in both implementations. While the running time for the heap based implementation increased linearly with the increase of land cells, the running time for the pure-MATLAB implementation, however, increased quadratically. For example, with the smallest sub-DEM, which has 197,000 land cells, the running time for the heap based and pure-MATLAB implementation are 38 and 84 seconds, respectively. The heap based implementation is only twice faster than the pure-MATLAB implementation. However, when the sub-DEM with 8 million land cells was used, the running time for the heap based and pure-MATLAB implementations are 0.47 and 15 hours, respectively, a difference of over 30 times. This comparison clearly indicated the importance of the data structures used in implementing the method.

We calculated inundation height for the entire North Carolina DEM using both binary heap and Fibonacci heap based implementations. The raster layer has more than 46 million land cells. The binary heap implementation ran in 5,821 seconds and the Fibonacci heap version ran in 50,358 seconds. Since both heap implementations used the same hash table, the computational complexity of the two heaps (Table 1) would suggest that the results should be the opposite of what they really were. However, there are two reasons for these results. First, as shown by Goldberg and Tarjan (1996), the constant factor for the extract minimum operation of the Fibonacci heap is much greater than that of the binary heap, thus making the operation more computationally expensive for the Fibonacci heap. Second, much of the performance gains found by using a Fibonacci heap over a binary heap are due to the efficiency of the Reduce Key operation. However, the binary heap outperforms the Fibonacci heap in calculating inundation height because the Reduce Key operations did

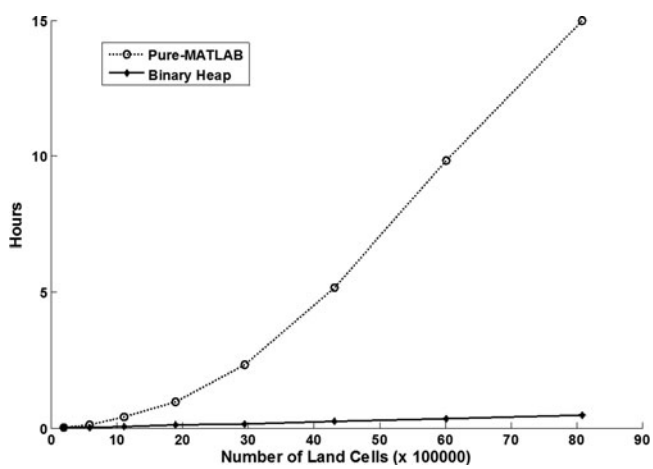

Figure 9. Running time comparison between a binary-heap-based implementation and a pure-MATLAB implementation.

not take place. This is true due to the function used to calculate inundation height and the property of Dijkstra's algorithm. If a low-cell has an inundation height of IH_{lc} and one of its neighbors has an inundation height of IH_{nbr} and an elevation of E_{nbr}, and already exists in the inundation front, then either the relationship $E_{nbr} \leq IH_{nbr} = IH_{lc}$ or $IH_{lc} < IH_{nbr} = E_{nbr}$ must be true. When the low-cell is extracted from the front, the new inundation height (IH'_{nbr}) of the neighbor is calculated, using Eq. (1) since $IH'_{nbr} = \max(IH_{lc}, E_{nbr})$. No matter which relationship ($E_{nbr} \leq IH_{nbr} = IH_{lc}$ or $IH_{lc} < IH_{nbr} = E_{nbr}$) is true, IH'_{nbr} is always equal to its existing inundation height (IH_{nbr}) in the front. Therefore, the neighbor's inundation height in the front will never need to be reduced. These factors eliminated the benefit of using a Fibonacci heap for this particular application.

A hash table was used to create an index for the cells in the inundation front to quickly locate them in the minimum priority queue. In addition to hash table, there are several other data structures which can be used to search the cells in the front (Li et al. 2005). However, with the observation that the inundation heights of front cells are never reduced, our binary-heap-based implementation can be further improved. One possibility is to remove the hash table entirely and flag the front cells on the output raster layer using some special value, for example, a negative inundation height. This way, the algorithm can check the output raster to see whether the neighbors of the low-cell are already in the front. Only the neighbors which are not in the front will be added and no other action is needed.

The first method was implemented as a model tool in Esri ArcGIS and the ModelBuilder diagram was shown in Figure 2. The method can also be implemented in any raster GIS software with the cost distance function available. The second method was implemented using C^{++} and MATLAB and can be accessed through MATLAB. We are considering implementing the second method using the Python programming language and making it available in Esri ArcGIS as a Python script tool in the future.

One important limitation of the existing inundation delineation methods, including the two methods developed here, is that they can only handle single DEM raster layer. As higher spatial resolution DEMs become increasingly available and are commonly distributed as DEM tiles, creating a single DEM raster layer for inundation calculation is not feasible. The process of creating a single raster layer from DEM tiles is tedious and the size of the DEM can easily exceed computer memory, which makes the calculation impossible if the methods are not disk-based (which means entire raster layers have to be loaded into memory in order to perform the calculation). Even if we can process single large DEM, the output raster layers usually have to be tiled, more often than not, to distribute the results. We are currently developing parallelized inundation delineation methods which can handle tiled DEMs and also take the advantages of multicore and cluster computing systems.

As indicated in the introduction, delineation of inundation from SLR differs from short-term and pulsed-flood inundation modeling which depends on surface parameters and the characteristics of individual storm events. Inundation from SLR, instead, reflects long-term and near steady-state flooding, which completely ignores storm effects and the possible changes of storm characteristics caused by SLRs.

Conclusions

One of the key requirements in the assessments of present and future impacts of SLR is the delineation of potentially inundated areas by different SLR scenarios. This research developed two methods for delineating potential inundation by SLRs. The methods require

a DEM and current ocean as the inputs and can be used to delineate inundation for different types of coastal landscapes and for any SLRs as long as the DEM's vertical resolutions are comparable with the SLR scenarios.

Both methods consider water connectivity and performed better than the simple "bathtub" approach. The error (overestimate of inundation) from the bathtub method, based on the North Carolina DEM, is the largest with SLRs below 1 m, ranging from 48% to 1.5% with an average of 17%. With SLRs above 1 m, the average error is only 0.6%. The methods, therefore, make the biggest difference in low SLRs, which are the most likely future projections according to the IPCC (2007).

The first method is based on the cost distance GIS function and was implemented using the Esri ArcGIS software. The method, like other existing methods, however, can only delineate the inundation by a specific SLR. Inspired by the cost distance function, we also developed an innovative method which calculates the minimum SLR (i.e., inundation height) to inundate each cell in a DEM. The importance of data structures used in implementing this method is clearly demonstrated in our research. While the running time of a binary heap based implementation increased linearly with the increase of DEM size, the running time of a non-heap implementation increased quadratically. The min-heap implementation outperformed the non-heap implementation significantly, especially with large DEMs. The Fibonacci heap data structure has a better theoretical computational complexity than the binary heap. However, the Fibonacci heap based implementation did not perform well because the Reduce Key operation is not necessary in this particular application.

Several improvements can be further explored ranging from using a better location index for searching the cells in the inundation front to removing the location index entirely. More importantly, all the existing methods, including the two we developed, are limited to handle single DEM. We need parallelized methods which can handle tiled DEMs and take the advantages of multicore and cluster computing systems.

Acknowledgements

Constructive criticism and comments by the guest editors and anonymous referees are greatly appreciated.

Funding

This research was supported by a grant (DE-FC02-06ER64298) from the U.S. Department of Energy through the National Institute for Climatic Change Research – Coastal Center at Tulane University.

References

Bindschadler, R. A. 1998. Future of the west Antarctic ice sheet. *Science* 282:428–429.

Cormen, T. H., C. E. Leiserson, R. L. Rivest, and C. Stein, 2001. *Introduction to algorithms*, 2nd ed. Cambridge, MA: MIT Press.

Craft, C., J. Clough, J. Ehman, S. Joye, R. Park, S. Pennings, H. Guo, and M. Machmuller, 2009. Forecasting the effects of accelerated sea-level rise on tidal marsh ecosystem services. *Frontiers in Ecology and the Environment* 7:73–78.

Dijkstra, E. W. 1959. A note on two problems in connexion with graphs. *Numerische Mathematik* 1:269–271.

Fredman, M. L., and R. Tarjan, 1987. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the Association for Computing Machinery* 34:596–615.

Goldberg, A. V., and R. Tarjan, 1996. Expected performance of Dijkstra's shortest path algorithm. *NEC Research Institute Report*, No. TR-530-96.

Hopkinson, C. S., A. E. Lugo, M. Alber, A. P. Covich, and S. J. van Bloem, 2008. Forecasting effects of sea-level rise and windstorms on coastal and inland ecosystems. *Frontiers in Ecology and the Environment* 6:255–263.

Intergovernmental Panel on Climate Change (IPCC). 2007. *Climate change 2007: The physical science basis*. Cambridge: Cambridge University Press.

Legra, L., X. Li, and A. T. Peterson, 2008. Biodiversity consequences of sea level rise in New Guinea. *Pacific Conservation Biology* 13:191–199.

Li, X., M. L. Larson, and B. R. Arthur, 2005. Creating buffers on surfaces. *Cartography and Geographic Information Science* 32:195–210.

Li, X., R. J. Rowley, J. C. Kostelnick, D. Braaten, J. Meisel, and K. Hulbutta, 2009. GIS analysis of global inundation impacts from sea level rise. *Photogrammetric Engineering and Remote Sensing* 75:807–818.

Marks, K., and P. Bates, 2000. Integration of high-resolution topographic data with floodplain flow models. *Hydrological Processes* 14:2109–2122.

Menon, S., J. Soberón, X. Li, and A. T. Peterson, 2010. Preliminary global assessment of terrestrial biodiversity consequences of sea level rise mediated by climate change. *Biodiversity and Conservation* 19:1599–1609.

Nicholls, R. J., N. Marinova, J. A. Lowe, S. Brown, P. Vellinga, D. de Gusmao, J. Hinkel, and R. S. J. Tol, 2011. Sea-level rise and its possible impacts given a 'beyond 4C world' in the twenty-first century. *Philosophical Transactions of the Royal Society A* 369:161–181.

Nicholls, R. J., and R. S. J. Tol, 2006. Impacts and responses to sea-level rise: A global analysis of the SRES scenarios over the twenty-first century. *Philosophical Transactions of the Royal Society A* 364:1073–1095.

Overpeck, J. T., B. L. Otto-Bliesner, G. H. Miller, D. R. Muhs, R. B. Alley, and J. T. Kiehl, 2006. Paleoclimatic evidence for future ice-sheet instability and rapid sea-level rise. *Science* 311:1747–1750.

Peterson, A. T., X. Li, and A. G. Navarro-Sigüenza, 2010. Joint effects of marine intrusion and climate change on the Mexican avifauna. *Annals of the Association of American Geographers* 100:908–916.

Poulter, B., and P. N. Halpin, 2008. Raster modeling of coastal flooding from sea-level rise. *International Journal of Geographical Information Science* 22:167–182.

Rignot, E., and P. Kanagaratam, 2006. Changes in the velocity structure of the Greenland Ice Sheet. *Science* 311:986–990.

Thomas, R., E. Rignot, G. Casassa, P. Kanagaratnam, C. Acuña, T. Akins, H. Brecher, E. Frederick, P. Gogineni, W. Krabill, S. Manizade, H. Ramamoorthy, A. Rivera, R. Russell, J. Sonntag, R. Swift, J. Yungel, and J. Zwally, 2004. Accelerated sea-level rise from West Antarctica. *Science* 306:255–258.

Virah-Sawmy, M., K. J. Willis, and L. Gillson, 2009. Threshold response of Madagascar's littoral forest to sea-level rise. *Global Ecology and Biogeography* 18:98–110.

Copyright of Marine Geodesy is the property of Taylor & Francis Ltd and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.